

Week-5 DEMO REPORT

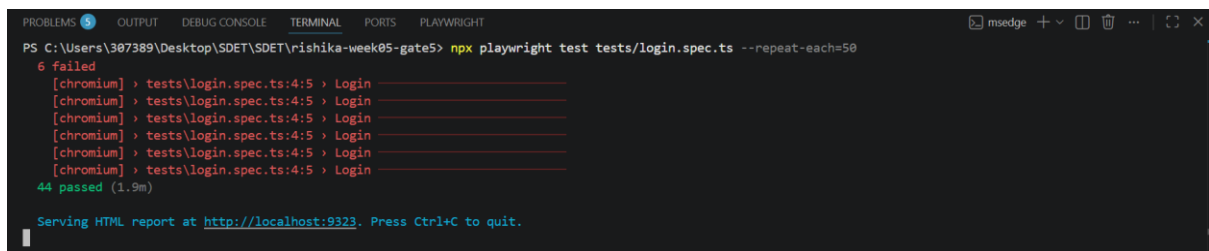
1.Flaky test:

Why this is flaky

- 100 ms is an unrealistically short timeout.
- If the dropdown takes 120 ms, 150 ms, or 200 ms to become ready, the test fails.
- On a fast machine it may pass.
- On a slower browser it may fail.

Before fix:

The login test occasionally failed while selecting the country.



```
PS C:\Users\307389\Desktop\SDET\SDET\rishika-week05-gate5> npx playwright test tests/login.spec.ts --repeat-each=50
6 failed
[chromium] › tests\login.spec.ts:4:5 › Login
[chromium] › tests\login.spec.ts:4:5 › Login
[chromium] › tests\login.spec.ts:4:5 › Login
[chromium] › tests\login.spec.ts:4:5 › Login
[chromium] › tests\login.spec.ts:4:5 › Login
[chromium] › tests\login.spec.ts:4:5 › Login
44 passed (1.9m)
Serving HTML report at http://localhost:9323. Press Ctrl+C to quit.
```

Q Search tests

All50✓ Passed44✗ Failed6Flaky0Skipped0🔄⚙️

Project: chromium

4/7/2026, 3:07:25 pm Total time: 1.9m

▼ login.spec.ts

✗ Loginchromium

login.spec.ts:4 ▶️ 📄 View Trace

5.1s

✗ Loginchromium

login.spec.ts:4 ▶️ 📄 View Trace

4.6s

✗ Loginchromium

login.spec.ts:4 ▶️ 📄 View Trace

4.5s

✗ Loginchromium

login.spec.ts:4 ▶️ 📄 View Trace

3.7s

✗ Loginchromium

login.spec.ts:4 ▶️ 📄 View Trace

5.1s

✗ Loginchromium

login.spec.ts:4 ▶️ 📄 View Trace

4.9s

✓ Loginchromium

login.spec.ts:4

4.1s

✓ Loginchromium

login.spec.ts:4

4.3s

Test Steps		
Filter steps		
> ✓ Before Hooks		1.9s
> ✓ Navigate to "/login" — login.spec.ts:6		787ms
> ✓ Fill "customer@example.com" getByLabel('Email') — login.spec.ts:8		401ms
> ✓ Fill "Password@123" getByLabel('Password') — login.spec.ts:9		385ms
> ✗ Select option getByLabel('Country') — login.spec.ts:10		165ms
> ✓ After Hooks		2.1s
> ✓ Worker Cleanup		641ms



SDET Retail Automation Lab
UST Global training app

[Home](#)
[Login](#)
[Sync Lab](#)
[Profile](#)
[Products](#)
[Frames](#)
[A11y Lab](#)
[Debug](#)
[POS](#)
[Returns](#)
[Cart](#)

[Checkout](#)
[Orders](#)
[API Docs](#)
[Admin](#)

DAY 3 DELAYED BUTTON ENABLED

Sign in to Retail Lab

Use the classroom credentials to practice labels, roles, form inputs, assertions, and synchronization.

Email

Password

Country

☐ Remember me

Sign in

Classroom Credentials

CUSTOMER
customer@example.com / Password@123

DECK DEMO
user@test.com / Secret123

LOCKED
locked@example.com / Password@123

Attachments	
	logs.ndjson
	error-context
 stdout <pre>[INFO] 2026-07-04T09:37:46.216Z - test started [INFO] 2026-07-04T09:37:48.246Z - test finished {"status":"failed"}</pre>	

After fix:

Removed the timeout and allowed Playwright's built-in auto-wait mechanism to wait until the dropdown was ready.

Evidence:

```
PS C:\Users\307389\Desktop\SDET\SDET\Fishika-week05-gates> npx playwright test tests/login.spec.ts --repeat-each=50
[INFO] 2026-07-04T09:45:40.454Z - test finished {"status":"passed"}
[INFO] 2026-07-04T09:45:40.577Z - test finished {"status":"passed"}

50 passed (1.4m)

To open last HTML report run:

npx playwright show-report
```

Test Steps

Filter steps

> ✓ Before Hooks1.6s

> ✓ Navigate to "/login" — login.spec.ts:6848ms

> ✓ Fill "customer@example.com" getByLabel('Email') — login.spec.ts:8104ms

> ✓ Fill "Password@123" getByLabel('Password') — login.spec.ts:993ms

> ✓ Select option getByLabel('Country') — login.spec.ts:1060ms

> ✓ Click getByRole('button', { name: 'Sign in' }) — login.spec.ts:121.1s

> ✓ After Hooks850ms

Attachments

stdout

[INFO] 2026-07-04T09:44:21.462Z - test started

[ERROR] 2026-07-04T09:44:22.580Z - flacky test

[INFO] 2026-07-04T09:44:23.651Z - test finished {"status":"passed"}

Conclusion:

All executions passed successfully. Playwright trace confirmed the dropdown was ready before interaction. The test depended on a fixed timeout instead of waiting for the UI to be ready. This caused intermittent failures on slower machines or under higher load.

2.Cart-Total:

Before Fix:

Symptom

Refund amount could be incorrect due to rounding differences when monetary values are calculated using decimal (floating-point) numbers.

Root Cause

Using floating-point arithmetic for money calculations can introduce precision and rounding errors, resulting in incorrect refund totals.

Evidence:

Verify cart total is calculated using exact paise values

cart-total.spec.ts:4 View Trace | 1.2s

chromium

Run

Errors

Error: expect(received).toBe(expected) // Object.is equality

Copy prompt

Expected: 99900
Received: undefined

10 |
11 | // Verify individual amounts are stored in paise
> 12 | expect(order.lineAmountPaise).toBe(99900);
13 |
14 | expect(order.taxPaise).toBe(4995);
15 | expect(order.totalPaise).toBe(104895);
at C:\Users\307389\Desktop\SDET\SDET\rishika-week05-gate5\tests\cart-total.spec.ts:12:35

Test Steps

Filter steps	
> ✓ Before Hooks	1.7s
> ✓ POST "/api/refund-lab/orders" — ./src/api/refund.api.ts:14	33ms
> ✓ Expect "toBeTruthy" — ./src/api/refund.api.ts:32	0ms
> ✗ Expect "toBe" — cart-total.spec.ts:12	2ms
> ✓ After Hooks	549ms
> ✓ Worker Cleanup	873ms

order.json

Copy

```
{  "id": 7042,  "orderNumber": "RET-7042",  "ownerKey": "user-1",  "placedOn": "2026-06-29",  "daysAgo": 5,  "returnWindowDays": 30,  "paymentMethod": "Original card",  "lines": [    {      "sku": "TEE",      "name": "Training Tee",      "unitPaise": 33300,      "qty": 3,      "refundedQty": 0,      "finalSale": false,      "nonReturnable": false    }  ],  "lineTotalPaise": 99900,  "taxPaise": 4995,  "totalPaise": 104895,  "refundableBalancePaise": 104895,  "refunds": [],  "refundCount": 0,  "lastRefund": null,  "status": "PAID"}
```

[logs.ndjson](#)
[error-context](#)

After Fix:

Verified all refund calculations using **integer paise** instead of decimal values.

Assertions were added to validate the refund amount, line amount, tax amount, and tax distribution.

Verification

- Verified refund line amount.
- Verified refund tax amount.
- Verified total refund amount.
- Verified tax shares add up to the original tax amount.

Evidence:

Verify refund calculations use exact paise values

cart-total.spec.ts:4

1.1s

chromium

✓ Run

Test Steps

Filter steps

> ✓ Before Hooks	1.2s
> ✓ POST "/api/refund-lab/orders" — ./src/api/refund.api.ts:14	34ms
> ✓ Expect "toBeTruthy" — ./src/api/refund.api.ts:32	0ms
> ✓ Expect "toBe" — cart-total.spec.ts:16	0ms
> ✓ Expect "toHaveLength" — cart-total.spec.ts:18	0ms
> ✓ Expect "toBe" — cart-total.spec.ts:19	0ms
> ✓ Expect "toBe" — cart-total.spec.ts:20	0ms
> ✓ Expect "toBe" — cart-total.spec.ts:30	0ms

Attachments

order.json

```
{
  "id": 7043,
  "orderNumber": "RET-7043",
  "ownerKey": "user-1",
  "placedOn": "2026-06-29",
  "daysAgo": 5,
  "returnWindowDays": 30,
  "paymentMethod": "Original card",
  "lines": [
    {
      "sku": "TEE",
      "name": "Training Tee",
      "unitPaise": 33300,
      "qty": 3,
      "refundedQty": 0,
      "finalSale": false,
      "nonReturnable": false
    }
  ],
  "lineTotalPaise": 99900,
  "taxPaise": 4995,
  "totalPaise": 104895,
  "refundableBalancePaise": 104895,
  "refunds": [],
  "refundCount": 0,
  "lastRefund": null,
  "status": "PAID"
}
```

```
ledger.json
{
  "id": 7043,
  "orderNumber": "RET-7043",
  "ownerKey": "user-1",
  "placedOn": "2026-06-29",
  "daysAgo": 5,
  "returnWindowDays": 30,
  "paymentMethod": "Original card",
  "lines": [
    {
      "sku": "TEE",
      "name": "Training Tee",
      "unitPaise": 33300,
      "qty": 3,
      "refundedQty": 0,
      "finalSale": false,
      "nonReturnable": false
    }
  ],
  "lineTotalPaise": 99900,
  "taxPaise": 4995,
  "totalPaise": 104895,
  "refundableBalancePaise": 104895,
  "refunds": [],
  "refundCount": 0,
  "lastRefund": null,
  "status": "PAID"
}

stdout
[INFO] 2026-07-04T09:58:58.965Z - test started
[INFO] 2026-07-04T09:58:58.968Z - Creating order
[INFO] 2026-07-04T09:58:59.028Z - Ledger verified {"refundableBalance":104895}
[INFO] 2026-07-04T09:58:59.034Z - test finished {"status":"passed"}
```

Conclusion:

- Refund calculations are performed using integer paise.
- No floating-point rounding issues.
- Refund amount matches the expected value exactly.
- Tax distribution is verified correctly.

3. Sync test hangs on WebKit:

Before fix:

Symptom

Offline sync test hangs or times out while opening the POS page.

Root Cause

The browser was switched to offline mode **before** the application finished loading.

Evidence:

Errors

Error: page.goto: net::ERR_INTERNET_DISCONNECTED at http://localhost:5173/pos

Copy prompt

Call log:

- navigating to "http://localhost:5173/pos", waiting until "load"

```
9 |  
10 | // Open POS while online  
> 11 | await page.goto("/pos");  
    |           ^  
12 |  
13 | // Wait until the page is fully loaded  
14 | await expect(page.getByRole("heading", { name: "POS" })).toBeVisible();  
    |                                     at C:\Users\307389\Desktop\SDET\SDET\rishika-week05-gate5\tests\pos.spec.ts:11:14
```

Test Steps

Filter steps

> ✓ Before Hooks	1.2s
> ✓ Set offline mode — pos.spec.ts:8	2ms
> ✗ Navigate to "/pos" — pos.spec.ts:11	54ms
> ✓ After Hooks	487ms

Screenshots



Press space to play

Try:

- Checking the network cables, modem, and router
- Reconnecting to Wi-Fi
- Running Windows Network Diagnostics

ERR_INTERNET_DISCONNECTED

Attachments

logs.ndjson

error-context

stdout

```
[INFO] 2026-07-04T10:15:49.912Z - test started  
[INFO] 2026-07-04T10:15:50.075Z - test finished {"status":"failed"}
```



After Fix:

Fix

Loaded the POS page completely before switching the browser to offline mode.

Proof

- Test passes consistently.
- No timeout while loading the page.
- Works across Chromium, Firefox, and WebKit.

```
Serving HTML Report at http://localhost:9323. Press Ctrl+C to quit.
PS C:\Users\307389\Desktop\SDET\SDET\rishika-week05-gate5> npx playwright test tests/pos.spec.ts --headed

Running 1 test using 1 worker

  ✓ 1 [chromium] > tests\pos.spec.ts:3:5 > Offline sync - load page before going offline (1.9s)
[INFO] 2026-07-04T10:20:08.747Z - test started
[INFO] 2026-07-04T10:20:09.356Z - POS page loaded successfully
[INFO] 2026-07-04T10:20:09.375Z - Application switched to offline mode
[INFO] 2026-07-04T10:20:09.388Z - Application reconnected successfully
[INFO] 2026-07-04T10:20:09.390Z - test finished {"status":"passed"}

 1 passed (4.8s)

To open last HTML report run:

npx playwright show-report
```

Q Filter steps

> ✓ Before Hooks1.2s

> ✓ Navigate to "/pos" — pos.spec.ts:10563ms

> ✓ Expect "toBeVisible" getByRole('heading', { name: 'POS' }) — pos.spec.ts:1336ms

> ✓ Set offline mode — pos.spec.ts:182ms

> ✓ Expect "toContainText" getByTestId('network-banner') — pos.spec.ts:2113ms

> ✓ Set offline mode — pos.spec.ts:261ms

> ✓ Expect "toContainText" getByTestId('network-banner') — pos.spec.ts:2810ms

> ✓ After Hooks702ms

Attachments

stdout

```
[INFO] 2026-07-04T10:20:08.747Z - test started
[INFO] 2026-07-04T10:20:09.356Z - POS page loaded successfully
[INFO] 2026-07-04T10:20:09.375Z - Application switched to offline mode
[INFO] 2026-07-04T10:20:09.388Z - Application reconnected successfully
[INFO] 2026-07-04T10:20:09.390Z - test finished {"status":"passed"}
```

4. Refund Posts Twice

Symptom

Submitting the same refund request twice created two refund records.

Root Cause

A new **Idempotency-Key** was generated for every refund request. Since each request had a different key, the backend treated them as separate refund requests.

Evidence:

Refund is created twice without idempotency key

refund.spec.ts:12 chromium View Trace | 1.4s

✖ Run

Errors

Error: expect(received).toBe(expected) // Object.is equality

Expected: 1
Received: 2

```
48 |  
49 | // This assertion FAILS because refundCount becomes 2  
> 50 | expect(ledger.refundCount).toBe(1);  
    |                                     ^  
51 |  
52 | });  
    at C:\Users\307389\Desktop\SDET\SDET\rishika-week05-gate5\tests\refund.spec.ts:50:32
```

Copy prompt

Test Steps

Filter steps

> ✓ Before Hooks	1.4s
> ✓ POST "/api/refund-lab/orders" — ./src/api/refund.api.ts:14	39ms
> ✓ Expect "toBeTruthy" — ./src/api/refund.api.ts:32	1ms
> ✓ POST "/api/refunds" — ./src/api/refund.api.ts:43	7ms
> ✓ Expect "toBeTruthy" — refund.spec.ts:29	0ms
> ✓ POST "/api/refunds" — ./src/api/refund.api.ts:43	4ms
> ✓ Expect "toBeTruthy" — refund.spec.ts:39	0ms
> ✓ GET "/api/refund-lab/orders/7045" — ./src/api/refund.api.ts:91	4ms
> ✓ Expect "toBeTruthy" — ./src/api/refund.api.ts:98	0ms
> ✖ Expect "toBe" — refund.spec.ts:50	2ms
> ✓ After Hooks	563ms
> ✓ Worker Cleanup	1.1s

```

    "refundCount": 2,
    "lastRefund": {
      "refundId": 8025,
      "orderId": 7045,
      "ownerKey": "user-1",
      "idempotencyKey": "refund-cf5101a8-aaaf-4e3f-b490-e1f83b1dfd36",
      "amountPaise": 34965,
      "lineAmountPaise": 33300,
      "taxPaise": 1665,
      "taxShares": [
        1665,
        1665,
        1665
      ],
      "paymentMethod": "Original card",
      "status": "REFUNDED",
      "createdAt": "2026-07-04T10:29:43.626Z"
    },
    "status": "PARTIALLY_REFUNDED"
  }
}

```

[logs.ndjson](#)

[error-context](#)

▼ [stdout](#)

```

[INFO] 2026-07-04T10:29:43.565Z - test started
[INFO] 2026-07-04T10:29:43.566Z - Creating order
[INFO] 2026-07-04T10:29:43.637Z - Ledger after duplicate refunds {"refundCount":2}
[INFO] 2026-07-04T10:29:43.745Z - test finished {"status":"failed"}

```

After Fix:

Fix:

Generated a single **Idempotency-Key** and reused it for both refund requests. This allowed the backend to identify the second request as a duplicate and return the existing refund instead of creating a new one.

Verification

- Verified both responses returned the same refundId.
- Verified the refund amount remained unchanged.
- Verified only one refund was recorded in the ledger.

Duplicate refund request should create only one refund using the same Idempotency-Key

refund.spec.ts:12

1.2s

[chromium](#)

✓ Run

▼ Test Steps

Q Filter steps

> ✓ Before Hooks	1.3s
> ✓ POST "/api/refund-lab/orders" — ./src/api/refund.api.ts:14	33ms
> ✓ Expect "toBeTruthy" — ./src/api/refund.api.ts:32	0ms
> ✓ POST "/api/refunds" — ./src/api/refund.api.ts:43	4ms
> ✓ Expect "toBeTruthy" — refund.spec.ts:32	0ms
> ✓ POST "/api/refunds" — ./src/api/refund.api.ts:43	4ms

▼ Attachments

▼ order.json

```
{
  "id": 7046,
  "orderNumber": "RET-7046",
  "ownerKey": "user-1",
  "placedOn": "2026-06-29",
  "daysAgo": 5,
  "returnWindowDays": 30,
  "paymentMethod": "Original card",
  "lines": [
    {
      "sku": "TEE",
      "name": "Training Tee",
      "unitPaise": 33300,
      "qty": 3,
      "refundedQty": 0,
      "finalSale": false,
      "nonReturnable": false
    }
  ],
  "lineTotalPaise": 99900,
  "taxPaise": 4995,
  "totalPaise": 104895,
  "refundableBalancePaise": 104895,
  "refunds": [],
  "refundCount": 0,
  "lastRefund": null,
  "status": "PAID"
}
```

▼ refund.json

```
{
  "refundId": 8026,
  "orderId": 7046,
  "ownerKey": "user-1",
  "idempotencyKey": "refund-1e503427-662f-45fc-bc88-9a69eabdf035",
  "amountPaise": 34965,
  "lineAmountPaise": 33300,
  "taxPaise": 1665,
  "taxShares": [
    1665,
    1665,
    1665
  ],
  "paymentMethod": "Original card",
  "status": "REFUNDED",
  "createdAt": "2026-07-04T10:30:37.878Z",
  "orderStatus": "PARTIALLY_REFUNDED",
  "refundCount": 1,
  "refundableBalancePaise": 69930
}
```

▼ replayRefund.json

```
{
  "refundId": 8026,
  "orderId": 7046,
  "ownerKey": "user-1",
  "idempotencyKey": "refund-1e503427-662f-45fc-bc88-9a69eabdf035",
  "amountPaise": 34965,
  "lineAmountPaise": 33300,
  "taxPaise": 1665,
  "taxShares": [
    1665,
    1665,
    1665
  ],
  "paymentMethod": "Original card",
  "status": "REFUNDED",
  "createdAt": "2026-07-04T10:30:37.878Z",
  "replayed": true
}
```

ledger.json

```
{
  "id": 7046,
  "orderNumber": "RET-7046",
  "ownerKey": "user-1",
  "placedOn": "2026-06-29",
  "daysAgo": 5,
  "returnWindowDays": 30,
  "paymentMethod": "Original card",
  "lines": [
    {
      "sku": "TEE",
      "name": "Training Tee",
      "unitPaise": 33300,
      "qty": 3,
      "refundedQty": 1,
      "finalSale": false,
      "nonReturnable": false
    }
  ],
  "lineTotalPaise": 99900,
  "taxPaise": 4995,
  "totalPaise": 104895,
  "refundableBalancePaise": 69930,
  "refunds": [
    {
      "refundId": 8026,
      "orderId": 7046,
      "ownerKey": "user-1",
      "idempotencyKey": "refund-1e503427-662f-45fc-bc88-9a69eabdf035",
      "amountPaise": 34965,
      "lineAmountPaise": 33300,
      "taxPaise": 1665,
      "taxShares": [
        1665,
        1665,
        1665
      ],
      "paymentMethod": "Original card",
      "status": "REFUNDED",
      "createdAt": "2026-07-04T10:30:37.878Z"
    }
  ],
  "refundCount": 1,
  "lastRefund": {
    "refundId": 8026,
    "orderId": 7046,
    "ownerKey": "user-1",
    "idempotencyKey": "refund-1e503427-662f-45fc-bc88-9a69eabdf035",
    "amountPaise": 34965,
    "lineAmountPaise": 33300,
    "taxPaise": 1665,
    "taxShares": [
      1665,
      1665,
      1665
    ],
    "paymentMethod": "Original card",
    "status": "REFUNDED",
    "createdAt": "2026-07-04T10:30:37.878Z"
  },
  "status": "PARTIALLY_REFUNDED"
}
```

stdout

```
[INFO] 2026-07-04T10:30:37.834Z - test started
[INFO] 2026-07-04T10:30:37.837Z - Creating order
[INFO] 2026-07-04T10:30:37.899Z - Verified idempotent refund {"refundCount":1}
[INFO] 2026-07-04T10:30:37.910Z - test finished {"status":"passed"}
```

Conclusion:

Duplicate refund requests no longer create multiple refunds. Ledger remains consistent with **refundCount = 1** and The same refund response is returned when the same **Idempotency-Key** is used.